

二级缓存解决热key问题

采用一套“防御性懒加载两级缓存”架构来解决热key问题，核心思路不是主动识别和针对热key进行特殊处理，而是通过架构设计让所有的key都能获得本地缓存保护，从而化解热key压力。

具体手段包括：

- 1.懒加载的两级缓存架构：使用Caffeine本地缓存作为第一级缓存，Redis分布式缓存作为第二级。所有请求先查本地缓存，未命中再查Redis，实现了按需加载的缓存保护。
- 2.空对象模式防缓存穿透：对于查询不存在的key，本地缓存中设置特殊标记的空对象，防止大量请求穿透到Redis和数据库，这是针对“不存在热key”的有效防护。
- 3.差异化过期与刷新策略：通过MCC动态配置不同缓存类型的过期时间和刷新频率，避免大量key同时过期导致“缓存雪崩”效应。
- 4.异步刷新机制：使用Caffeine的RefreshAfterWrite配置，在数据写入后异步刷新，避免同步加载导致性能瓶颈。
- 5.容量限制与智能淘汰：Caffeine内部使用W-TinyLFU算法，结合LRU和LFU优势，智能识别和保留高频访问的key，自然形成热key保护。
- 6.监控与动态降级：通过Cat监控缓存命中率、加载时间等指标，结合MCC配置中心实现动态开关和参数调整，具备线上应急能力。

第一部分：热key的判断标准和监控机制

1.1 热key的定义标准

根据美团内部Squirrel缓存系统的定义：**热key的判定标准：**

- **默认阈值：**单key的QPS超过5000，或者流量超过10MB/s
- **阈值可配置：**可以通过告警配置进行修改
- **监控平台：**通过Squirrel管理平台的热点key监控功能查看

1.2 多层监控体系

系统通过三层监控来识别热key：**第一层：Squirrel分布式缓存监控**

- **平台监控：**Squirrel管理平台的热点key监控功能
- **实时告警：**当单key QPS超过5000或流量超过10MB/s时触发P2级别告警
- **趋势分析：**提供热点key占比趋势图，查看指定时间区间内读/写/流量热点key的情况

第二层：Caffeine本地缓存统计监控 系统通过Caffeine的 `recordStats()` 功能收集详细的缓存统计信息：

第三层：CAT实时监控上报 系统通过CAT监控上报详细的缓存指标：

第二部分：热key的防护措施和本地缓存架构

2.1 两级缓存架构设计

系统采用"本地Caffeine缓存 + 分布式Redis缓存"的两级架构来专门应对热key问题：**架构优势：**

1. **本地缓存（L1）**：Caffeine，访问速度极快（纳秒级）
2. **分布式缓存（L2）**：Squirrel（美团内部的Redis），数据共享
3. **分级防护**：热key在本地缓存命中，避免冲击分布式缓存

2.2 空对象模式防止缓存穿透

对于热key可能导致的缓存穿透问题，系统实现了空对象模式：

空对象模式的作用：

1. **防止缓存穿透**：对不存在的key也缓存空对象
2. **减少数据库压力**：避免大量请求直接打到数据库
3. **可配置开关**：通过MCC动态控制是否启用空对象模式

2.3 差异化缓存配置

系统为不同类型的缓存设置了差异化的配置：

不同缓存的配置差异：

- `skuBase`：12秒访问过期，15秒刷新
- `citySku`：不同的过期和刷新时间
- `spuBase`、`brand`、`recipe` 等都有各自的配置

设计目的：

1. **避免同时过期**：不同缓存类型错开过期时间
2. **按业务特点配置**：不同数据的重要性不同
3. **动态可调**：通过MCC可以实时调整配置

第三部分：refreshAfterWrite的刷新机制

3.1 refreshAfterWrite的核心原理

`refreshAfterWrite` 是Caffeine提供的异步刷新机制，其核心特点是：**基于写入时间计时：**

- 计时从数据**写入缓存的时间**开始
- 后续的读取访问**不会重置**刷新计时器
- 名称中的"Write"明确表示了基于写入时间

3.2 刷新触发机制

刷新触发条件：

1. **时间条件**：距离上次写入时间超过 `refreshAfterWrite` 配置的时间
2. **访问条件**：下次访问该key时触发刷新
3. **异步执行**：刷新操作在后台线程池中执行

刷新流程：

3.3 与expireAfterAccess的协同工作

系统同时配置了两种策略来实现智能的缓存管理：

```
.expireAfterAccess(cacheExpireSecondsAfterAccess, TimeUnit.SECONDS)
// 清理冷数据
.refreshAfterWrite(cacheRefreshSeconds, TimeUnit.SECONDS)
// 刷新热数据
```

协同效果：

数据类型	expireAfterAccess	refreshAfterWrite	最终效果
热数据	频繁访问，不断重置过期时间	定期刷新，保持数据新鲜	长期驻留，定期更新
温数据	偶尔访问，可能过期	定期刷新	可能被清理，刷新时重新加载
冷数据	长期不访问，必然过期	无意义（已被清理）	被清理，释放内存

3.4 防止大量key同时刷新的机制

问题：如果有大量key同时过期，会导致大量并发刷新请求**解决方案**：

1. **差异化配置**：不同缓存类型设置不同的刷新时间
2. **基于写入时间**：每个key的刷新时间点不同
3. **异步刷新**：刷新操作不阻塞当前请求
4. **线程池限制**：通过线程池控制并发刷新数量

第四部分：线程池和并发控制

4.1 专用线程池配置

系统为本地缓存刷新配置了专用线程池：

```
this.executorService = ExecutorServices.forThreadPoolExecutor("local-cache-load-executor");
```

4.2 线程池的并发控制作用

线程池的主要作用：

1. **限制并发数**：控制同时进行的刷新任务数量
2. **队列缓冲**：当并发数超过限制时，任务进入队列等待
3. **资源隔离**：缓存刷新任务与业务逻辑线程隔离
4. **防止雪崩**：避免大量并发刷新导致服务崩溃

4.3 美团内部的线程池配置

查看系统中的线程池配置：

4.4 防止大量key同时刷发的完整防护体系

多层防护机制：

1. **第一层：Caffeine内置机制**
 - `refreshAfterWrite`：异步刷新，不阻塞请求
 - 基于写入时间：每个key刷新时间点不同
2. **第二层：线程池限制**
 - 专用线程池：`local-cache-load-executor`
 - 并发数限制：控制同时刷新的任务数量
 - 队列缓冲：超过限制的任务进入队列
3. **第三层：差异化配置**
 - 不同缓存类型：不同的刷新时间
 - 避免同时过期：错开刷新时间点
4. **第四层：监控降级**
 - CAT监控：实时监控刷新性能
 - MCC配置：动态调整刷新策略
 - 降级机制：刷新失败时的降级处理

